# TRABAJO PRÁCTICO: FLUJOS DE TRABAJO COLABORATIVO

*Unidad 4 – Semana 2*

Metodología de Sistemas II – Tecnicatura Universitaria en Programación a Distancia – UTN

| | |
|---|---|
| **Apellido y nombre:** | Figueroa, Ignacio – DNI 45.406.120 |
| **Comisión:** | C4 - A26 |

## Parte 1: Estrategias de branching

### 1. GitFlow vs. Trunk-based Development

**GitFlow** usa varias ramas de larga duración: main (lo que está en producción), develop (integración), y ramas feature, release y hotfix. **Trunk-based** usa una sola rama principal (trunk o main), y todos integran ahí cambios chicos con frecuencia, al menos una vez por día. Si hay ramas, duran horas o pocos días.

| Factor | GitFlow | Trunk-based |
|---|---|---|
| Estructura de comunicación | Jerárquica: cada uno trabaja aislado en su rama y la integración pasa por puntos de control (develop, release), muchas veces a cargo de un responsable. | Sincronizada: todos integran en la misma rama todo el tiempo, así que el equipo tiene que comunicarse seguido y tener mucha disciplina. |
| Deuda de integración | Alta: las ramas pueden quedar separadas semanas. Cuanto más tiempo pasa, más se alejan de develop y más cuesta integrarlas. | Baja: como se integra todos los días, las diferencias entre lo de cada uno son chicas y se resuelven enseguida. |
| Conflictos de fusión (Merge Hell) | Más probables y más grandes: al final se juntan muchos cambios de muchas personas a la vez. | Pocos y chicos: los conflictos aparecen apenas se generan, cuando todavía son fáciles de resolver. |

En resumen, GitFlow da más control pero integra tarde, y Trunk-based integra rápido pero exige buenos tests automáticos y que el equipo se coordine bien.

### 2. Caso de estudio: software médico

Recomendaría **GitFlow**. Las razones:

- **Control:** al ser un sistema crítico con auditorías estrictas, conviene tener puntos de control claros. La rama release permite congelar una versión, probarla, auditarla y aprobarla formalmente antes de pasar a main.
- **Estabilidad:** main solo tiene versiones aprobadas y probadas. Lo nuevo vive en develop y no llega a producción hasta que se libera.
- **Ciclo de lanzamientos largo:** si se publica cada tres meses, no hace falta integrar a producción todos los días. GitFlow está pensado justamente para versiones planificadas.
- **Equipo grande y jerárquico:** encaja con un modelo donde los responsables aprueban qué entra en cada versión.
- **Hotfixes:** si aparece un error grave en producción, la rama hotfix permite corregirlo sin esperar al próximo lanzamiento.

La contra es la deuda de integración, así que igual conviene que las ramas feature sean cortas y se actualicen seguido con develop.

### 3. Feature Flags

Una Feature Flag es un "interruptor" en el código, generalmente un if que consulta una configuración, que decide si una funcionalidad se muestra o no. Por ejemplo:

```
if flags.esta_activa("exportar_pdf"):
    mostrar_boton_exportar_pdf()
```

Así, un programador puede subir a main la funcionalidad aunque no esté terminada, porque queda **apagada** y el usuario final no la ve. El código se integra todos los días y se evitan las ramas largas, pero la funcionalidad recién se "enciende" cuando está lista, sin hacer un deploy nuevo. También sirve para activarla primero solo para algunos usuarios, o para apagarla rápido si da problemas. Lo importante es borrar la flag cuando la funcionalidad ya está estable, para no acumular código muerto.

## Parte 2: Procesos de integración colaborativa

### 4. El Pull Request como herramienta de comunicación

Una buena descripción de PR debería tener:

- **Título claro** que diga qué hace el cambio.
- **Issue relacionado** (por ejemplo "Closes #12"), para saber de dónde viene la tarea.
- **Qué cambia y por qué:** el problema que resuelve y la solución elegida, en pocas líneas.
- **Cómo probarlo:** los pasos para verificar que funciona.
- **Evidencias:** capturas de pantalla si cambia algo visual, o la salida de los tests.
- **Impacto o riesgos:** si cambia la base de datos, si rompe algo que usan otros o si hay que hacer algo especial al desplegar.
- **Checklist:** tests agregados, documentación actualizada, etc.

Con eso, el revisor entiende el contexto antes de leer el código y puede concentrarse en lo importante.

### 5. La cultura de la code review

Los errores de sintaxis los detectan el linter y los tests. La revisión de código sirve para cosas que una herramienta no ve:

- **Transferencia de conocimiento:** quien revisa aprende cómo funciona esa parte del sistema y quien escribió recibe sugerencias. Así el conocimiento no queda en una sola persona; si alguien se va o se enferma, otro puede seguir.
- **DRY (Don't Repeat Yourself):** el revisor puede darse cuenta de que algo ya existe en otra parte del proyecto y evitar que se duplique lógica, que después hay que mantener en varios lugares.
- **SOLID:** se revisa el diseño, por ejemplo que una clase no haga demasiadas cosas (responsabilidad única) o que se pueda extender sin modificar lo que ya funciona (abierto/cerrado). Esto hace que el código sea más fácil de mantener a largo plazo.
- **Calidad y consistencia:** que el código sea legible, siga las convenciones del equipo y no tenga problemas de seguridad o de rendimiento.

La revisión tiene que ser sobre el código y no sobre la persona, con comentarios respetuosos y concretos.

### 6. Estrategias de merge y el historial

| Estrategia | Qué hace | Cómo queda el grafo |
|---|---|---|
| Create a Merge Commit (historial verídico) | Mantiene todos los commits de la rama y agrega un commit de merge con dos padres. | Muestra exactamente cómo pasaron las cosas, con ramas que se separan y se unen. Es el más fiel, pero con muchos PR se vuelve difícil de leer. |
| Squash and Merge (historial limpio) | Junta todos los commits de la rama en uno solo y lo agrega a main. | Un commit por PR, sin los "fix", "test", etc. Muy fácil de leer, pero se pierde el detalle de cómo se fue haciendo. |
| Rebase and Merge (historial lineal) | Reaplica los commits de la rama, uno por uno, arriba de main, sin commit de merge. | Una línea recta, sin bifurcaciones. Se conservan los commits individuales, pero se reescriben (cambian sus hashes). |

Ejemplo con una rama de tres commits (A, B y C) integrada a main:

```
Merge commit:            Squash and merge:      Rebase and merge:
*   M  Merge rama       * S  Feature (A+B+C)   * C'
|\                       * 2                    * B'
| * C                     * 1                    * A'
| * B                                            * 2
| * A                                            * 1
|/
* 2
* 1
```

## Parte 3: Estandarización y documentación

### 7. CONTRIBUTING.md

El integrante nuevo debería encontrar:

- **Cómo preparar el entorno:** requisitos, cómo instalar dependencias y cómo correr el proyecto y los tests.
- **Flujo de ramas:** qué estrategia se usa y cómo nombrar las ramas (por ejemplo feature/12-login).
- **Convención de commits:** el formato de los mensajes (por ejemplo "feat: agrega login" o "fix: corrige total").
- **Estilo de código:** formatter y linter que se usan y cómo correrlos.
- **Cómo hacer un Pull Request:** plantilla, cuántas aprobaciones hacen falta y qué tiene que pasar en el pipeline.
- **Definition of Done:** qué se necesita para considerar una tarea terminada (tests, cobertura, documentación).
- **Dónde pedir ayuda:** a quién preguntar y por qué canal.

### 8. Architecture Decision Records (ADR)

Un ADR es un documento corto que registra **una decisión técnica importante**: el contexto (qué problema había), la decisión que se tomó, las alternativas que se analizaron y las consecuencias (lo bueno y lo malo). Por ejemplo: "ADR-003: Usar PostgreSQL en lugar de MongoDB".




Para alguien que se suma un año después, el código muestra **qué** se hizo pero no **por qué**. Sin el ADR, esa persona podría pensar que una decisión fue un error y cambiarla, sin saber que en su momento se descartó la alternativa por una buena razón. Con el ADR entiende el contexto de ese momento y, si cambia algo, puede escribir un ADR nuevo que reemplace al anterior. Así queda la historia de las decisiones y no solo la del código.

### 9. Plantillas de Issues y Pull Requests

- **Información completa desde el principio:** la plantilla pide los datos necesarios (pasos para reproducir un bug, qué se esperaba, etc.), así no hay que andar preguntando.
- **Uniformidad:** todos los issues y PR tienen la misma estructura, entonces se leen y se revisan más rápido.
- **Menos olvidos:** los checklists recuerdan cosas como agregar tests o actualizar la documentación.
- **Mejor organización:** facilita clasificar, priorizar y buscar las tareas.

## Parte 4: Validación de estándares locales

### 10. Git Hooks y la "primera línea de defensa"

Un Git Hook es un script que Git ejecuta automáticamente cuando pasa algo, por ejemplo antes de hacer un commit (pre-commit) o antes de un push (pre-push). Husky es una herramienta del mundo JavaScript que facilita configurar estos hooks y compartirlos con todo el equipo dentro del repositorio.

Por ejemplo, un hook pre-commit puede correr el linter y el formatter sobre los archivos modificados. Si hay errores, el hook termina con error y **Git cancela el commit**. Así el código que no cumple el estándar ni siquiera se commitea, y mucho menos llega al repositorio compartido.

Es importante validar en la máquina del desarrollador porque:

- **Es más rápido y barato:** el error se ve en segundos, sin esperar a que corra el pipeline en el servidor.
- **No se ensucia el repositorio compartido** con commits que después hay que arreglar.
- **Las revisiones de código** se concentran en la lógica y no en detalles de formato.

Igual, como un hook local se puede saltear (git commit --no-verify), los mismos controles se repiten en el pipeline de CI como segunda línea de defensa.